

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Queue”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 9 April 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	1
BAB II PEMBAHASAN.....	3
2.1 Pengertian Queue	3
2.2 Implementasi Stack pada Java.....	4
2.2.1 Class Queue Array (QueryArray_2511533016).....	4
2.2.2 Class Queue Array Driver (QueueArrayDriver_2511533016) ..	6
2.2.3 Class Queue Linked List (QueueLinkedList _2511533016).....	8
2.2.4 Class Reverse Date (ReverseDate_2511533016).....	10
2.2.5 Class Iterasi Queue (IterasiQueue_2511533016).....	11
BAB III PENUTUP	14
3.1 Kesimpulan.....	14
3.2 Saran	15
DAFTAR PUSTAKA.....	16
LAPORAN TUGAS PRAKTIKUM	17
1. Class Antrian Locket (AntrianLocket_2511533016)	17
2. Class Antrian Locket Driver (AntrianLocketDriver_2511533016)	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data menjadi hal yang sangat penting, terutama ketika data harus diproses secara teratur dan berurutan. Salah satu struktur data yang digunakan untuk mengatur alur data tersebut adalah **Queue (antrian)**. Queue bekerja dengan prinsip **FIFO (First In First Out)**, yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Konsep ini banyak dijumpai dalam kehidupan sehari-hari, seperti antrian di kasir, antrian kendaraan di lampu merah, hingga sistem pelayanan pelanggan. Dalam konteks pemrograman, Queue digunakan untuk menangani proses yang membutuhkan urutan eksekusi yang adil dan terstruktur, seperti pada sistem operasi (penjadwalan proses), jaringan komputer (pengiriman paket data), dan aplikasi berbasis event. Dengan memahami dan mengimplementasikan Queue, programmer dapat membuat sistem yang lebih efisien dalam mengelola data yang masuk dan keluar secara teratur.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 4 tentang Queue adalah sebagai berikut:

1. Memahami konsep dasar struktur data Queue beserta prinsip kerjanya (FIFO). Mengetahui operasi dasar pada Stack seperti push, pop, dan peek.
2. Mampu mengimplementasikan Queue menggunakan berbagai cara seperti array maupun ArrayList.
3. Melatih kemampuan logika dan algoritma dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 4 tentang Queue adalah sebagai berikut:

1. Pengelolaan data yang terstruktur atau Data diproses secara berurutan sesuai urutan masuk.
2. Efisiensi dalam sistem antrian yaitu Cocok untuk sistem pelayanan, seperti antrean pelanggan atau tiket.
3. Memudahkan simulasi dunia nyata, karena konsepnya sesuai dengan kehidupan sehari-hari.
4. Meningkatkan performa program, terutama dalam menangani data yang datang secara terus-menerus (streaming data).

BAB II

PEMBAHASAN

2.1 Pengertian Queue

Queue atau antrian merupakan salah satu **struktur data linear** dalam bidang Ilmu Komputer yang digunakan untuk mengelola sekumpulan data dengan prinsip **FIFO (First In First Out)**, yaitu elemen yang pertama kali dimasukkan ke dalam struktur akan menjadi elemen pertama yang dikeluarkan. Konsep ini meniru sistem antrian dalam kehidupan nyata, seperti antrean di bank, rumah sakit, atau kasir, di mana individu yang datang lebih awal akan dilayani terlebih dahulu dibandingkan yang datang belakangan.

Secara teknis, Queue memiliki dua ujung utama, yaitu **front (depan)** sebagai titik penghapusan data dan **rear (belakang)** sebagai titik penambahan data. Proses memasukkan data ke dalam queue disebut **enqueue**, sedangkan proses mengeluarkan data disebut **dequeue**. Selain itu, terdapat operasi tambahan seperti **peek** atau **front** untuk melihat elemen terdepan tanpa menghapusnya. Struktur ini memastikan bahwa urutan data tetap terjaga sesuai dengan waktu kedatangannya, sehingga sangat cocok digunakan dalam sistem yang membutuhkan keadilan dan keteraturan dalam pemrosesan data.

Dalam implementasinya, Queue dapat dibangun menggunakan berbagai pendekatan, seperti array maupun linked list. Pada implementasi berbasis array, queue biasanya memiliki kapasitas tertentu sehingga diperlukan pengelolaan indeks untuk menghindari kondisi penuh (overflow) atau kosong (underflow). Sementara itu, pada implementasi berbasis linked list, queue bersifat dinamis karena ukuran dapat bertambah atau berkurang sesuai kebutuhan. Selain queue linear, terdapat juga variasi lain seperti circular queue, priority queue, dan deque (double-ended queue) yang dikembangkan untuk mengatasi keterbatasan atau menyesuaikan kebutuhan tertentu dalam pemrosesan data.

Queue memiliki peran yang sangat penting dalam berbagai sistem komputer modern. Dalam sistem operasi, queue digunakan untuk mengatur penjadwalan proses (process scheduling) agar setiap proses mendapatkan giliran eksekusi secara adil. Dalam jaringan komputer, queue berfungsi untuk mengatur

aliran paket data yang dikirim dan diterima agar tidak terjadi tabrakan atau kehilangan data. Selain itu, queue juga digunakan dalam sistem antrian printer, manajemen request pada server web, serta berbagai aplikasi real-time lainnya yang memerlukan pengolahan data secara berurutan.

Dengan demikian, Queue tidak hanya sekadar konsep dasar dalam struktur data, tetapi juga merupakan komponen fundamental yang mendukung efisiensi dan keteraturan dalam berbagai aplikasi pemrograman. Pemahaman yang baik mengenai Queue akan membantu pengembang perangkat lunak dalam merancang sistem yang mampu menangani data secara sistematis, efisien, dan sesuai dengan urutan prioritas pemrosesan yang telah ditentukan.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 4 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Queue, di bawah ini kode program yang dituliskan pada praktikum pekan 4.

2.2.1 Class Queue Array (QueryArray_2511533016)

```

1 package Pekan4_2511533016;
2
3 public class QueueArray_2511533016 {
4     int front_3016, rear_3016, size_3016;
5     int capacity_3016;
6     int array_3016[];
7
8     public QueueArray_2511533016(int capacity_3016) {
9         this.capacity_3016 = capacity_3016;
10        front_3016 = this.size_3016 = 0;
11        array_3016 = new int [this.capacity_3016];
12    }
13
14    boolean isFull(QueueArray_2511533016 queue) {
15        return (queue.size_3016 == queue.capacity_3016);
16    }
17
18    boolean isEmpty(QueueArray_2511533016 queue) {
19        return (queue.size_3016 == 0);
20    }
21
22    void enqueue(int item_3016) {
23        if (isFull(this))
24            return;
25        this.rear_3016 = (this.rear_3016 + 1) % this.capacity_3016;
26        this.array_3016[this.rear_3016] = item_3016;
27        this.size_3016 = this.size_3016 + 1;
28        System.out.println(item_3016 + " enqueue to queue");
29    }
30
31    int dequeue_3016() {
32        if (isEmpty(this))
33            return Integer.MIN_VALUE;
34        int item_3016 = this.array_3016[this.front_3016];
35        this.front_3016 = (this.front_3016 + 1) % this.capacity_3016;
36        this.size_3016 = this.size_3016 - 1;
37        return item_3016;
38    }
39
40    int front_3016() {
41        if (isEmpty(this))
42            return Integer.MIN_VALUE;
43        return this.array_3016[this.front_3016];
44    }
45
46    int rear_3016() {
47        if (isEmpty(this))
48            return Integer.MIN_VALUE;
49        return this.array_3016[this.rear_3016];
50    }
51
52    void display_3016() {
53        int i;
54        if (front_3016 == rear_3016) {
55            System.out.println("\nAntrian Kosong \n");
56            return;
57        }
58
59        //Jumlah dari belakang dan cetak
60        for (i = front_3016; i < rear_3016; i++) {
61            System.out.print(" %d <== ", array_3016[i]);
62        }
63    }
64
65 }
66
67
68
69

```

Gambar 2.2.1

Program Java tersebut merupakan implementasi **struktur data Queue (antrian)** menggunakan pendekatan **array melingkar (circular queue)** yang bertujuan untuk mengelola data secara FIFO (First In First Out). Kelas utama yang digunakan adalah `QueueArray_2511533016`, yang berfungsi sebagai representasi dari queue itu sendiri.

Pada bagian awal program, terdapat deklarasi atribut atau variabel instance yaitu `front_3016`, `rear_3016`, dan `size_3016` yang masing-masing digunakan untuk menandai posisi elemen terdepan, posisi elemen terakhir, serta jumlah elemen yang ada di dalam queue. Selain itu, terdapat variabel `capacity_3016` yang menyimpan kapasitas maksimum queue, serta array `array_3016[]` yang berfungsi sebagai tempat penyimpanan elemen-elemen queue.

Selanjutnya, konstruktor `QueueArray_2511533016(int capacity_3016)` digunakan untuk menginisialisasi queue. Pada bagian ini, nilai kapasitas ditentukan berdasarkan parameter yang diberikan, kemudian `front_3016` dan `size_3016` diatur ke 0 sebagai kondisi awal queue kosong. Array juga dibuat dengan ukuran sesuai kapasitas yang ditentukan.

Program kemudian menyediakan method `isFull()` yang berfungsi untuk mengecek apakah queue sudah penuh, yaitu dengan membandingkan nilai `size_3016` dengan `capacity_3016`. Jika sama, maka queue tidak dapat menerima elemen baru. Sebaliknya, method `isEmpty()` digunakan untuk mengecek apakah queue kosong dengan melihat apakah `size_3016` bernilai 0.

Proses penambahan elemen dilakukan oleh method `enqueue(int item_3016)`. Pada method ini, pertama dilakukan pengecekan apakah queue penuh. Jika tidak, maka nilai `rear_3016` akan diperbarui menggunakan operasi modulo $(rear + 1) \% capacity$, yang merupakan ciri khas circular queue agar indeks kembali ke awal ketika mencapai batas array. Setelah itu, elemen dimasukkan ke dalam array dan ukuran queue (`size_3016`) ditambah satu. Program juga menampilkan pesan bahwa elemen berhasil dimasukkan.

Sedangkan proses penghapusan elemen dilakukan oleh method `dequeue_3016()`. Method ini akan terlebih dahulu mengecek apakah queue kosong. Jika kosong, maka akan mengembalikan nilai `Integer.MIN_VALUE` sebagai indikator error. Jika tidak kosong, elemen pada posisi `front_3016` akan

diambil, kemudian posisi front digeser menggunakan operasi modulo yang sama seperti pada enqueue. Setelah itu, ukuran queue dikurangi satu dan elemen yang dihapus dikembalikan.

Untuk melihat elemen tanpa menghapusnya, tersedia method `front_3016()` dan `rear_3016()`. Method `front_3016()` mengembalikan elemen paling depan, sedangkan `rear_3016()` mengembalikan elemen paling belakang. Keduanya juga melakukan pengecekan apakah queue kosong sebelum mengakses array. Terakhir, method `display_3016()` digunakan untuk menampilkan isi queue. Jika kondisi `front == rear`, program menganggap queue kosong dan menampilkan pesan “Antrian Kosong”. Jika tidak, program melakukan iterasi dari indeks `front_3016` hingga `rear_3016` untuk mencetak setiap elemen dalam queue. Namun, perlu dicatat bahwa logika perulangan ini belum sepenuhnya menangani kondisi circular (ketika rear kembali ke depan), sehingga bisa menjadi titik perbaikan.

2.2.2 Class Queue Array Driver (QueueArrayDriver_2511533016)

```

1 package Pekan4_2511533016;
2
3 public class QueueArrayDriver_2511533016 {
4
5     public static void main(String[] args) {
6         QueueArray_2511533016 queue_3016 = new QueueArray_2511533016(1000);
7         queue_3016.enqueue(10);
8         queue_3016.enqueue(20);
9         queue_3016.enqueue(30);
10        queue_3016.enqueue(40);
11        System.out.println("Item di depan " + queue_3016.front_3016());
12        System.out.println("Item di paling belakang " + queue_3016.rear_3016());
13        System.out.println("Tampilan queue");
14        queue_3016.display_3016();
15        System.out.println();
16        System.out.println(queue_3016.dequeue_3016() + " dihapus dari queue ");
17        System.out.println("Item didepan: " + queue_3016.front_3016());
18        System.out.println("Item dibelakang: " + queue_3016.rear_3016());
19        System.out.println("Tampilan queue setelah satu data dihapus");
20        queue_3016.display_3016();
21    }
22 }
23
24 }

```

```

10 enqueue to queue
20 enqueue to queue
30 enqueue to queue
40 enqueue to queue
Item di depan 0
Item di paling belakang 40
Tampilan queue
0 <== 10 <== 20 <== 30 <==
0 dihapus dari queue
Item didepan: 10
Item dibelakang: 40
Tampilan queue setelah satu data dihapus
10 <== 20 <== 30 <==

```

Gambar 2.2.2

Program tersebut merupakan sebuah **kelas driver** bernama **QueueArrayDriver_2511533016** yang berfungsi untuk menguji implementasi

struktur data queue yang terdapat pada kelas **QueueArray_2511533016**. Pada bagian awal, program mendefinisikan package **Pekan4_2511533016** sebagai tempat pengelompokan kelas agar lebih terstruktur. Selanjutnya dideklarasikan kelas utama **QueueArrayDriver_2511533016** yang memiliki method **main(String[] args)** sebagai titik awal eksekusi program.

Di dalam method main, program pertama kali membuat sebuah objek queue dengan nama **queue_3016** menggunakan konstruktor dari kelas **QueueArray_2511533016** dengan kapasitas **1000**. Objek **queue_3016** inilah yang akan digunakan untuk menjalankan seluruh operasi queue.

Setelah objek dibuat, program melakukan proses penambahan data ke dalam queue menggunakan method **enqueue(int item_3016)**. Data yang dimasukkan adalah **10, 20, 30, dan 40**, yang dimasukkan secara berurutan ke dalam objek **queue_3016**. Proses ini menempatkan data pada bagian belakang (rear) antrian sesuai dengan konsep FIFO (First In First Out).

Selanjutnya, program menampilkan elemen yang berada di bagian depan dan bagian belakang queue menggunakan method **front_3016()** dan **rear_3016()**. Method **front_3016()** digunakan untuk mengambil nilai elemen terdepan dari **queue_3016**, sedangkan method **rear_3016()** digunakan untuk mengambil elemen paling belakang. Hasil dari kedua method tersebut ditampilkan menggunakan perintah output ke layar.

Setelah itu, program menampilkan seluruh isi queue dengan memanggil method **display_3016()** pada objek **queue_3016**. Method ini bertugas mencetak semua elemen dalam queue dari posisi depan hingga belakang sehingga pengguna dapat melihat susunan data secara keseluruhan.

Kemudian program melakukan operasi penghapusan data menggunakan method **dequeue_3016()**. Method ini akan menghapus dan mengembalikan elemen yang berada di posisi depan dari **queue_3016**. Nilai yang dihapus kemudian langsung ditampilkan ke layar sebagai informasi bahwa proses dequeue berhasil dilakukan.

Setelah proses penghapusan, program kembali memanggil method **front_3016()** dan **rear_3016()** untuk menampilkan kondisi terbaru dari elemen depan dan belakang queue. Perubahan ini menunjukkan bahwa elemen terdepan

telah bergeser ke data berikutnya setelah proses dequeue.

Pada bagian akhir, program kembali menampilkan seluruh isi queue dengan memanggil method **display_3016()** untuk kedua kalinya. Hal ini bertujuan untuk memperlihatkan kondisi queue setelah satu elemen dihapus sehingga dapat dibandingkan dengan kondisi sebelumnya.

2.2.3 Class Queue Linked List (QueueLinkedList_2511533016)

```

1 package Pekan4_2511533016;
2
3 import java.util.LinkedList;
4
5 public class QueueLinkedList_2511533016 {
6
7     public static void main(String[] args) {
8         Queue<Integer> q_3016 = new LinkedList<>();
9         // tambah elemen (0, 1, 2, 3, 4, 5) ke antrian
10         for(int i_3016 = 0; i_3016 < 6; i_3016++)
11             q_3016.add(i_3016);
12
13         // Menampilkan isi antrian
14         System.out.println("Elemen Antrian =" + q_3016);
15         // untuk menghapus kepala antrian
16         int hapus_3016 = q_3016.remove();
17         System.out.println("Hapus elemen =" + hapus_3016);
18         System.out.println(q_3016);
19         // untuk melihat antrian terdepan
20         int depan_3016 = q_3016.peek();
21         System.out.println("Kepala Antran =" + depan_3016);
22         // untuk melihat banyak antrian
23         int banyak_3016 = q_3016.size();
24         System.out.println("Size Antrian =" + banyak_3016);
25     }
26 }
27
28 }
29

```

```

Elemen Antrian =[0, 1, 2, 3, 4, 5]
Hapus elemen =0
[1, 2, 3, 4, 5]
Kepala Antran = 1
Size Antrian = 5

```

Gambar 2.2.3

Program tersebut merupakan implementasi sederhana **Queue menggunakan LinkedList** di Java yang ditulis dalam kelas **QueueLinkedList_2511533016** dan berfungsi untuk mendemonstrasikan operasi dasar antrian seperti menambah data, menghapus data, melihat elemen terdepan, serta mengetahui jumlah elemen.

Pada bagian awal, program mendefinisikan package **Pekan4_2511533016** sebagai tempat pengelompokan kelas. Kemudian terdapat perintah **import java.util.LinkedList;** yang digunakan untuk mengimpor kelas LinkedList dari library Java, yang nantinya akan dimanfaatkan sebagai implementasi dari struktur data queue.

Selanjutnya dideklarasikan kelas utama **QueueLinkedList_2511533016** yang memiliki method **main(String[] args)** sebagai titik awal eksekusi program. Di dalam method main, program membuat sebuah objek queue bernama **q_3016** dengan tipe **Queue<Integer>** yang diimplementasikan menggunakan **LinkedList**. Artinya, queue ini akan menyimpan data bertipe integer dan bersifat dinamis (tidak memiliki batas kapasitas tetap seperti array).

Setelah queue dibuat, program menambahkan elemen ke dalam antrian menggunakan perulangan **for**. Variabel **i_3016** digunakan sebagai counter yang berjalan dari 0 hingga kurang dari 6, sehingga nilai yang dimasukkan ke dalam queue adalah **0, 1, 2, 3, 4, dan 5**. Penambahan data dilakukan menggunakan method **add()**, yang secara otomatis menambahkan elemen ke bagian belakang queue.

Kemudian program menampilkan seluruh isi queue dengan mencetak objek **q_3016**, sehingga semua elemen yang telah dimasukkan akan terlihat dalam bentuk urutan antrian.

Setelah itu, program melakukan operasi penghapusan elemen menggunakan method **remove()**. Method ini akan menghapus elemen yang berada di bagian depan queue (kepala antrian) sesuai dengan prinsip FIFO. Nilai yang dihapus disimpan dalam variabel **hapus_3016** dan kemudian ditampilkan ke layar. Setelah penghapusan, isi queue kembali ditampilkan untuk menunjukkan perubahan yang terjadi.

Selanjutnya program mengambil elemen paling depan tanpa menghapusnya menggunakan method **peek()**. Nilai ini disimpan dalam variabel **depan_3016** dan ditampilkan sebagai “kepala antrian”. Method ini berguna untuk melihat elemen terdepan tanpa mengubah isi queue.

Terakhir, program menghitung jumlah elemen yang tersisa di dalam queue menggunakan method **size()**. Nilai tersebut disimpan dalam variabel **banyak_3016** dan ditampilkan sebagai ukuran atau jumlah elemen dalam antrian.

2.2.4 Class Reverse Date (ReverseDate_2511533016)

```

1 package Pekan4_2511533016;
2
3 import java.util.LinkedList;
4
5
6 public class ReverseDate_2511533016 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q_3016 = new LinkedList<Integer>();
10        q_3016.add(1);
11        q_3016.add(2);
12        q_3016.add(3);
13        System.out.println("Sebelum reverse" + q_3016);
14        Stack<Integer> s_3016 = new Stack<Integer>();
15        while (!q_3016.isEmpty()) {
16            s_3016.push(q_3016.remove());
17        }
18        while (!s_3016.isEmpty()) {
19            q_3016.add(s_3016.pop());
20        }
21        System.out.println("sesudah reverse = " + q_3016);
22    }
23 }
24
25
26
27
28

```

Sebelum reverse [1, 2, 3]
sesudah reverse = [3, 2, 1]

Gambar 2.2.4

Program tersebut merupakan implementasi manipulasi struktur data **Queue** dan **Stack** dalam Java yang bertujuan untuk **membalik urutan elemen (reverse)** dalam antrian. Program ini ditulis dalam kelas **ReverseDate_2511533016** dan memanfaatkan **LinkedList** sebagai implementasi queue serta **Stack** sebagai alat bantu pembalikan data.

Pada bagian awal, program mendefinisikan package **Pekan4_2511533016** sebagai tempat pengelompokan file. Kemudian terdapat perintah **import java.util.LinkedList;** yang digunakan untuk mengimpor kelas **LinkedList** sebagai implementasi queue. Meskipun tidak terlihat pada potongan import, program juga menggunakan struktur **Stack** dari library Java.

Selanjutnya dideklarasikan kelas **ReverseDate_2511533016** yang memiliki method utama **main(String[] args)** sebagai titik awal eksekusi program. Di dalam method ini, pertama-tama dibuat sebuah objek queue bernama **q_3016** dengan tipe **Queue<Integer>** yang diimplementasikan menggunakan **LinkedList**. Queue ini akan menyimpan data bertipe integer.

Setelah queue dibuat, program menambahkan beberapa elemen ke dalam antrian menggunakan method **add()**, yaitu nilai **1, 2, dan 3**. Data dimasukkan secara berurutan sehingga urutan awal queue adalah **[1, 2, 3]**. Kemudian program

menampilkan isi queue sebelum dilakukan proses pembalikan dengan perintah output, sehingga pengguna dapat melihat kondisi awal antrian.

Selanjutnya, program membuat sebuah objek stack bernama **s_3016** dengan tipe **Stack<Integer>**. Stack ini digunakan sebagai struktur bantu untuk membalik urutan data karena stack bekerja dengan prinsip **LIFO (Last In First Out)**.

Proses pembalikan dimulai dengan perulangan **while pertama** yang berjalan selama queue tidak kosong, menggunakan kondisi **!q_3016.isEmpty()**. Pada setiap iterasi, elemen dari queue diambil menggunakan method **remove()**, lalu langsung dimasukkan ke dalam stack menggunakan method **push()**. Proses ini memindahkan seluruh elemen dari queue ke stack, sehingga urutannya menjadi terbalik di dalam stack.

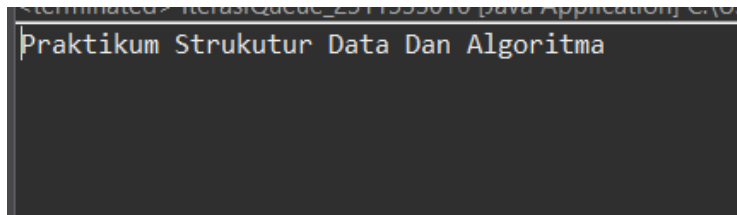
Setelah semua elemen berpindah ke stack, program menjalankan **perulangan while kedua** dengan kondisi **!s_3016.isEmpty()**. Pada tahap ini, elemen dari stack diambil menggunakan method **pop()**, kemudian dimasukkan kembali ke queue menggunakan method **add()**. Karena stack bersifat LIFO, elemen yang terakhir masuk ke stack akan keluar terlebih dahulu, sehingga ketika dimasukkan kembali ke queue, urutan elemen menjadi terbalik dari urutan semula. Terakhir, program menampilkan isi queue setelah proses reverse dengan perintah output. Hasilnya menunjukkan bahwa urutan awal [1, 2, 3] telah berubah menjadi [3, 2, 1].

2.2.5 Class Iterasi Queue (IterasiQueue_2511533016)

```

1 package Pekan4_2511533016;
2
3 import java.util.Iterator;
4
5
6
7 public class IterasiQueue_2511533016 {
8
9     public static void main(String[] args) {
10         Queue<String> q_3016 = new LinkedList<>();
11
12         q_3016.add("Praktikum");
13         q_3016.add("Struktur");
14         q_3016.add("Data");
15         q_3016.add("Dan");
16         q_3016.add("Algoritma");
17         Iterator<String> iterator_3016 = q_3016.iterator();
18         while (iterator_3016.hasNext()) {
19             System.out.print(iterator_3016.next() + " ");
20         }
21     }
22 }
23
24

```



Gambar 2.2.5

Program tersebut merupakan implementasi penggunaan **Queue dengan proses iterasi (perulangan)** di Java yang ditulis dalam kelas **IterasiQueue_2511533016**.

Program ini bertujuan untuk menampilkan seluruh elemen dalam queue menggunakan bantuan **Iterator**.

Pada bagian awal, program mendefinisikan package **Pekan4_2511533016** sebagai tempat pengelompokan kelas dalam satu proyek. Kemudian terdapat perintah **import java.util.Iterator;** yang digunakan untuk mengimpor kelas Iterator, yaitu interface yang berfungsi untuk melakukan penelusuran (iterasi) terhadap elemen-elemen dalam suatu koleksi. Meskipun tidak ditampilkan, program juga menggunakan LinkedList sebagai implementasi dari queue.

Selanjutnya dideklarasikan kelas utama **IterasiQueue_2511533016** yang memiliki method **main(String[] args)** sebagai titik awal eksekusi program. Di dalam method main, program membuat sebuah objek queue bernama **q_3016** dengan tipe **Queue<String>** yang diimplementasikan menggunakan LinkedList. Queue ini digunakan untuk menyimpan data bertipe String.

Setelah queue dibuat, program menambahkan beberapa elemen ke dalam antrian menggunakan method **add()**, yaitu string **"Praktikum"**, **"Struktur"**, **"Data"**, **"Dan"**, dan **"Algoritma"**. Data tersebut dimasukkan secara berurutan ke dalam queue sehingga urutan elemen tetap terjaga sesuai prinsip FIFO. Kemudian, program membuat sebuah objek iterator bernama **iterator_3016** dengan memanggil method **iterator()** pada objek **q_3016**. Iterator ini berfungsi untuk mengakses setiap elemen dalam queue satu per satu tanpa harus menghapusnya.

Selanjutnya, program menggunakan perulangan **while** dengan kondisi **iterator_3016.hasNext()**. Kondisi ini digunakan untuk mengecek apakah masih ada elemen berikutnya di dalam queue. Selama masih ada elemen, program akan terus menjalankan perulangan.

Di dalam perulangan, method **iterator_3016.next()** digunakan untuk

mengambil elemen berikutnya dari queue, kemudian elemen tersebut langsung ditampilkan menggunakan perintah output. Proses ini berlangsung hingga seluruh elemen dalam queue berhasil ditampilkan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan materi dan implementasi program yang telah dibuat, dapat disimpulkan bahwa **Queue (antrian)** merupakan struktur data linear yang bekerja dengan prinsip **FIFO (First In First Out)**, di mana elemen yang pertama masuk akan menjadi elemen pertama yang keluar. Konsep ini telah berhasil diimplementasikan dalam beberapa bentuk program, baik menggunakan **array (QueueArray_2511533016)** maupun **LinkedList (QueueLinkedList_2511533016)**.

Pada implementasi berbasis array, queue menggunakan konsep **circular queue** untuk mengoptimalkan penggunaan memori, dengan memanfaatkan indeks **front** dan **rear** yang berputar. Sementara itu, pada implementasi berbasis **LinkedList**, queue bersifat lebih dinamis karena tidak memiliki batas kapasitas tetap, sehingga lebih fleksibel dalam penyimpanan data.

Melalui program **driver (QueueArrayDriver_2511533016)**, dapat dilihat bagaimana operasi dasar queue seperti **enqueue**, **dequeue**, **front**, **rear**, dan **display** dijalankan dan diuji secara langsung. Program ini membuktikan bahwa queue dapat digunakan untuk mengelola data secara terstruktur dan berurutan. Selain itu, penggunaan **Iterator pada IterasiQueue_2511533016** menunjukkan cara mengakses seluruh elemen queue tanpa mengubah isinya, sedangkan program **ReverseDate_2511533016** membuktikan bahwa queue dapat dikombinasikan dengan struktur data lain seperti **stack** untuk melakukan operasi lanjutan, seperti membalik urutan data.

Secara keseluruhan, seluruh program yang dibuat telah menunjukkan bahwa Queue tidak hanya dipahami secara teori, tetapi juga dapat diimplementasikan dalam berbagai bentuk dan kebutuhan, sehingga memperkuat pemahaman terhadap cara kerja dan penerapannya dalam pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Perlu dilakukan pengembangan lebih lanjut pada program, seperti memperbaiki method display agar mendukung kondisi circular secara sempurna.
2. Sebaiknya menambahkan penanganan error (exception handling) untuk kondisi queue kosong (underflow) dan queue penuh (overflow) agar program lebih aman.
3. Gunakan penamaan variabel yang lebih sederhana dan konsisten supaya kode lebih mudah dibaca.
4. Penggunaan Queue dapat dikembangkan lebih lanjut dengan mengkombinasikannya dengan struktur data lain, seperti stack atau priority queue, untuk menyelesaikan permasalahan yang lebih kompleks.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Structures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS

PRAKTIKUM

SOAL:

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO First In First Out)

1. Class Antrian Loket (AntrianLoket_2511533016)

```
1 package Pekand_2511533016;
2 public class AntrianLoket_2511533016 {
3     int front_3016, rear_3016, size_3016;
4     int max_3016;
5     String queue_3016[];
6
7
8     public AntrianLoket_2511533016(int max_3016) {
9         this.max_3016 = max_3016;
10        front_3016 = -1;
11        rear_3016 = -1;
12        queue_3016 = new String [this.max_3016];
13    }
14
15    boolean isfull_3016(AntrianLoket_2511533016 queue_3016) {
16        return (queue_3016.rear_3016 == queue_3016.max_3016 - 1);
17    }
18
19    boolean isempty_3016(AntrianLoket_2511533016 queue_3016) {
20        return (queue_3016.front_3016 == -1);
21    }
22
23    void enqueue_3016(String namapelanggan_3016) {
24        if (isfull_3016(this))
25            return;
26        if (this.front_3016 == -1)
27            this.front_3016 = 0;
28        this.rear_3016 = (this.rear_3016 + 1) % this.max_3016;
29        this.queue_3016[this.rear_3016] = namapelanggan_3016;
30        System.out.println("Data berhasil ditambahkan ke antrian ");
31    }
32
33    String dequeue_3016() {
34        if (isempty_3016(this))
35            return null;
36        String namapelanggan_3016 = this.queue_3016[this.front_3016];
37        if (this.front_3016 == this.rear_3016) {
38            this.front_3016 = -1;
39            this.rear_3016 = -1;
40        } else {
41            this.front_3016 = (this.front_3016 + 1) % this.max_3016;
42        }
43        return namapelanggan_3016;
44    }
45
46    void display_3016() {
47        int number_3016 = 1;
48        if (isempty_3016(this)) {
49            System.out.println("Antrian Kosong");
50            return;
51        }
52
53        for (int i_3016 = front_3016; i_3016 <= rear_3016; i_3016++) {
54            System.out.println(number_3016 + ". " + queue_3016[i_3016]);
55        }
56        System.out.println();
57    }
58
59    String front_3016() {
60        if (isempty_3016 (this))
61            return null;
62        return this.queue_3016[this.rear_3016];
63    }
64
65    void reverse_3016() {
66        String[] temp = new String[max_3016];
67        int count = 0;
68
69        while (!isempty_3016(this)) {
70            temp[count++] = dequeue_3016();
71        }
72
73        for (int i = count - 1; i >= 0; i--) {
74            enqueue_3016(temp[i]);
75        }
76    }
77 }
78 }
```

Gambar 1

Kode program ini merupakan implementasi **Queue (Antrian) Melingkar (Circular Queue)** dalam bahasa Java yang digunakan untuk simulasi antrian loket, terdiri dari satu kelas bernama AntrianLoket_2511533016 dalam package

Pekan4_2511533016.

Pada bagian awal kelas, dideklarasikan beberapa atribut yaitu `front_3016` dan `rear_3016` sebagai penanda posisi depan dan belakang antrian, `size_3016` untuk ukuran aktual antrian, `max_3016` untuk kapasitas maksimum, serta array `queue_3016[]` bertipe String sebagai tempat penyimpanan nama pelanggan. Konstruktor kelas menerima parameter `max_3016` untuk menentukan kapasitas antrian, kemudian menginisialisasi `front` dan `rear` dengan nilai -1 sebagai tanda bahwa antrian masih kosong, sekaligus mengalokasikan array sesuai kapasitas yang diberikan.

Terdapat dua method pengecekan kondisi antrian, yaitu `isFull_3016` yang mengembalikan nilai true apabila posisi `rear` telah mencapai batas maksimum kapasitas, dan `isEmpty_3016` yang mengembalikan true apabila nilai `front` masih bernilai -1 yang berarti belum ada data dalam antrian. Method `enqueue_3016` berfungsi untuk menambahkan nama pelanggan baru ke dalam antrian. Jika antrian sudah penuh maka proses dibatalkan, jika antrian masih kosong maka `front` diset ke 0, kemudian posisi `rear` digeser menggunakan operasi modulo agar antrian bersifat melingkar, lalu nama pelanggan disimpan pada posisi tersebut. Method `dequeue_3016` berfungsi untuk mengambil dan menghapus pelanggan dari posisi terdepan antrian. Apabila antrian kosong maka dikembalikan nilai null, apabila hanya tersisa satu elemen maka `front` dan `rear` direset ke -1, dan apabila masih terdapat elemen lain maka `front` digeser maju secara melingkar menggunakan operasi modulo.

Method `display_3016` digunakan untuk menampilkan seluruh isi antrian dari posisi depan hingga belakang secara berurutan dengan nomor urut, dan apabila antrian kosong akan ditampilkan pesan "Antrian Kosong". Method `front_3016` digunakan untuk melihat data pelanggan yang berada di posisi terdepan tanpa menghapusnya dari antrian, dan mengembalikan null apabila antrian kosong. Terakhir, method `reverse_3016` berfungsi untuk membalik seluruh urutan antrian dengan cara mengeluarkan semua elemen ke dalam array sementara `temp` menggunakan `dequeue`, kemudian memasukkannya kembali ke antrian dalam urutan terbalik menggunakan `enqueue`, sehingga pelanggan yang semula berada di posisi paling belakang akan menjadi yang paling depan.

2. Class Antrian Loret Driver (AntrianLoretDriver_2511533016)

```

1 package Pekan4_2511533016;
2
3 import java.util.Scanner;
4
5 public class AntrianLoretDriver_2511533016 {
6
7     public static void main(String[] args) {
8         Scanner sc_3016 = new Scanner(System.in);
9         AntrianLoret_2511533016 queue_3016 = new AntrianLoret_2511533016(20);
10
11         int pilihan_3016;
12
13         do {
14             System.out.println("\n===== PROGRAM ANTRIAN LORET =====");
15             System.out.println("1. Tambah Antrian");
16             System.out.println("2. Hapus Antrian");
17             System.out.println("3. Tampilkan Antrian");
18             System.out.println("4. Reverse");
19             System.out.println("5. Keluar");
20             System.out.print("Pilih menu: ");
21             pilihan_3016 = sc_3016.nextInt();
22             sc_3016.nextLine(); // membersihkan buffer
23
24             switch (pilihan_3016) {
25                 case 1:
26                     if (queue_3016.isFull_3016(queue_3016)) {
27                         System.out.println("Antrian sudah penuh!");
28                     } else {
29                         System.out.print("Masukkan nama pelanggan : ");
30                         String nama_3016 = sc_3016.nextLine();
31                         queue_3016.enqueue_3016(nama_3016);
32                     }
33                     break;
34
35                 case 2:
36                     String keluar_3016 = queue_3016.dequeue_3016();
37                     if (keluar_3016 == null) {
38                         System.out.println("Antrian kosong, tidak ada yang dihapus!");
39                     } else {
40                         System.out.println(keluar_3016 + " telah dilayani");
41                     }
42                     break;
43
44                 case 3:
45
46                     void display_3016() {
47                         int number_3016 = 1;
48                         if (!isEmpty_3016(this)) {
49                             System.out.println("Antrian Kosong");
50                             return;
51                         }
52
53                         for (int i_3016 = front_3016; i_3016 <= rear_3016; i_3016++) {
54                             System.out.println(number_3016 + ". " + queue_3016[i_3016]);
55                             number_3016++;
56                         }
57                         System.out.println();
58                     }
59
60                     String front_3016() {
61                         if (isEmpty_3016(this))
62                             return null;
63                         return this.queue_3016[this.rear_3016];
64                     }
65
66                     void reverse_3016() {
67                         String[] temp = new String[max_3016];
68                         int count = 0;
69
70                         while (!isEmpty_3016(this)) {
71                             temp[count++] = dequeue_3016();
72                         }
73
74                         for (int i = count - 1; i >= 0; i--) {
75                             enqueue_3016(temp[i]);
76                         }
77                     }
78
79

```

Gambar 2

Kode program ini merupakan kelas driver bernama `AntrianLoretDriver_2511533016` yang berfungsi sebagai program utama untuk menjalankan sistem antrian loket secara interaktif melalui menu pilihan. Kelas ini menggunakan `java.util.Scanner` untuk membaca input dari pengguna melalui keyboard.

Pada bagian awal method `main`, dibuat sebuah objek `Scanner` bernama `sc_3016` untuk menangani input pengguna, serta sebuah objek antrian `queue_3016` bertipe `AntrianLoret_2511533016` dengan kapasitas maksimum 20 pelanggan. Variabel `pilihan_3016` bertipe integer dideklarasikan untuk menyimpan pilihan menu yang dimasukkan pengguna.

Program kemudian menjalankan struktur perulangan `do-while` yang akan

terus berjalan selama pengguna belum memilih opsi keluar. Di dalam perulangan tersebut, program menampilkan menu utama bertuliskan "PROGRAM ANTRIAN LOKET" dengan lima pilihan yaitu Tambah Antrian, Hapus Antrian, Tampilkan Antrian, Reverse, dan Keluar. Setelah menu ditampilkan, program meminta pengguna memasukkan pilihan menggunakan `nextInt()`, lalu membersihkan buffer input dengan `nextLine()`.

Pilihan pengguna kemudian diproses menggunakan struktur switch. Pada case 1 untuk menambah antrian, program terlebih dahulu memeriksa apakah antrian sudah penuh menggunakan method `isFull_3016`. Jika penuh, ditampilkan pesan "Antrian sudah penuh!", namun jika masih tersedia ruang, pengguna diminta memasukkan nama pelanggan yang kemudian ditambahkan ke antrian melalui method `enqueue_3016`. Pada case 2 untuk menghapus antrian, program memanggil method `dequeue_3016` untuk mengambil pelanggan dari depan antrian. Jika hasil yang dikembalikan bernilai null artinya antrian kosong dan ditampilkan pesan bahwa tidak ada yang dihapus, sedangkan jika berhasil maka ditampilkan nama pelanggan yang telah dilayani. Pada case 3 untuk menampilkan antrian, program langsung memanggil method `display_3016` yang akan mencetak seluruh isi antrian secara berurutan. Pada case 4 untuk membalik antrian, program memeriksa terlebih dahulu apakah antrian kosong menggunakan `isEmpty_3016`. Jika kosong, ditampilkan pesan bahwa antrian tidak bisa di-reverse, namun jika tidak kosong maka method `reverse_3016` dipanggil untuk membalik urutan antrian, diikuti pesan konfirmasi dan tampilan isi antrian hasil pembalikan. Pada case 5, program mencetak pesan "Terima kasih, program selesai!" sebagai tanda bahwa pengguna memilih untuk keluar. Selain kelima pilihan tersebut, blok default akan menampilkan pesan "Pilihan tidak valid!" apabila pengguna memasukkan angka di luar rentang menu.

Perulangan do-while akan berhenti ketika nilai pilihan_3016 sama dengan 5, dan setelah itu objek Scanner ditutup menggunakan `sc_3016.close()` untuk membebaskan sumber daya sebelum program berakhir.

OUTPUT UNTUK MENAMBAH :

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan : sofian
Data berhasil ditambahkan ke antrian

```

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan : fedo
Data berhasil ditambahkan ke antrian

```

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan : nabil
Data berhasil ditambahkan ke antrian

```

OUTPUT UNTUK MELIHAT ANTRIAN :

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
|
Isi Antrian :
1. sofian
2. fedo
3. nabil

```


OUTPUT UNTUK MEREVERSE ANTRIAN :

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
Data berhasil ditambahkan ke antrian
(Antrian berhasil di-reverse!)
1. nabil
2. fedo
3. sofian

```

OUTPUT UNTUK MENGHAPUS DIDEPAN DAN MELIHAT SETELAH DIHAPUS ANTRIAN :

```

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
nabil telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

Isi Antrian :
1. fedo
2. sofian

```